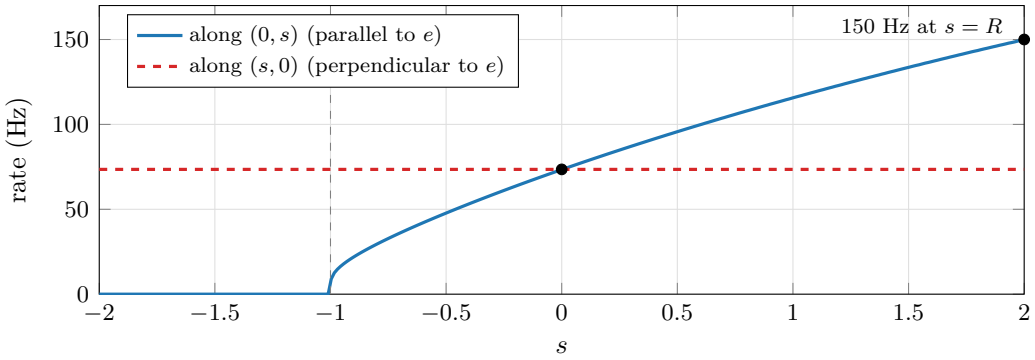
Practice Test 3B: Nengo & Simple Dynamics — Solutions

Total: 20 marks plus 2 bonus. Simulation data: `pn3_data/make_data_3b.py` (Nengo 4).

Question 1 (4)

(a) [2] A neuron only "sees" the part of $\mathbf{x}$ along its encoder, $\langle e, \mathbf{x}\rangle$. Nengo intercepts and max rates are in normalized units, so the neuron sees $\langle e, \mathbf{x}\rangle / R$, with $R = 2$.

- **Along** $(0, s)$**:** $\langle e, \mathbf{x}\rangle = s$. The neuron starts firing at $s/R = -0.5$, i.e. at $s = -1$, and rises with the LIF shape to 150 Hz at $s = R = 2$. Below $s = -1$ it is silent.
- **Along** $(s, 0)$**:** $\langle e, \mathbf{x}\rangle = 0$ for every s, so the current never changes. The rate is a **flat line** at the value of the first curve at $s = 0$. That value is positive because the intercept is negative: about 74 Hz with $\tau_{\text{ref}} = 2$ ms and $\tau_{RC} = 20$ ms (any value between 0 and 150 earns the mark).



Marking

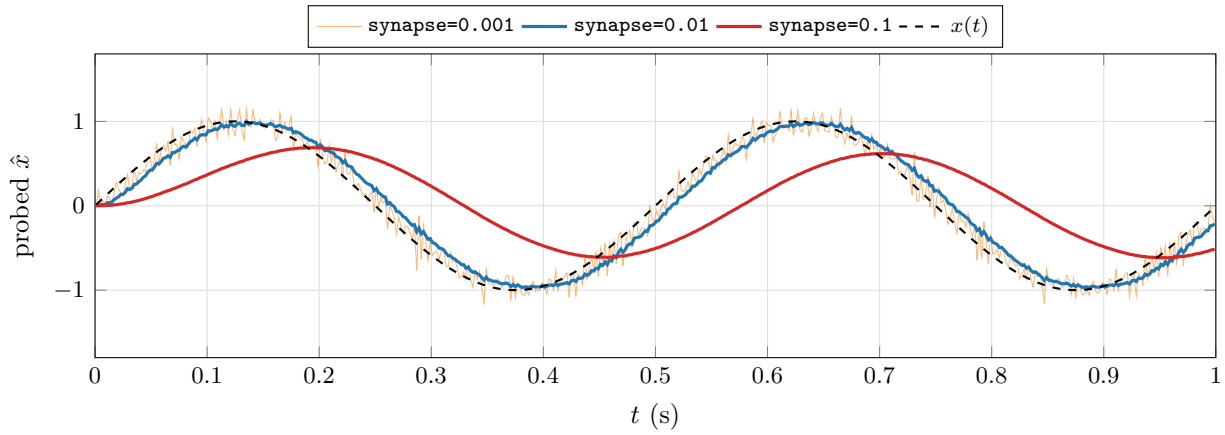
0.5: parallel curve is zero below $s = -1$. 0.5: it rises to 150 Hz at $s = 2$ (not at $s = 1$). 1: perpendicular line is constant and nonzero, matching the parallel curve at $s = 0$.

Where marks are usually lost

Putting the threshold at $s = -0.5$ (forgetting the radius), or drawing the perpendicular case as a second tuning curve. Movement perpendicular to e is invisible to this neuron.

(b) [2] The probe synapse only filters the *recorded* value. It is a measurement choice, so it does **not** change what the ensemble represents or anything downstream. Each probe is a low-pass filter on the decoded spikes, applied to a 2 Hz sine ($\omega = 4\pi$ rad/s):

- `0.001`: almost unfiltered, so the trace is **very noisy** (each spike shows up as a sharp jump) but has no visible lag.
- `0.01`: a little smoothing and a small lag. This gives the cleanest picture of the represented value.
- `0.1`: smooth, but **attenuated and delayed**. The gain is $1/\sqrt{1 + (\omega\tau)^2} = 1/\sqrt{1 + (0.4\pi)^2} \approx 0.62$ and the lag is $\arctan(0.4\pi)/\omega \approx 72$ ms.



Marking

0.5: small synapse is the noisiest. 0.5: large synapse is smoother, *smaller* and *later*. 1: the probe synapse does not change the representation (it only filters what is recorded).

Question 2 (4)

(a) [2]

- x: the value the population represents (decoded from its activity).
- $g(u)$: what the input connection computes. It is decoded from the input population with function decoders and arrives through the synapse.
- $f(x)$: what the recurrent connection computes. The population decodes $f(x)$ from its own activity and feeds it back to itself.
- τ: the synaptic time constant. It sets the timescale, because nothing reaches the population faster than the synapse allows.
- $-x$: this comes from the **synapse**, not from any connection. Everything arriving at the population is filtered: $x = h*(g(u)+f(x))$. In the Laplace domain this is $X = (G+F)/(1+s\tau)$, so $\tau sX = G+F-X$, i.e. $\tau\dot{x} = g(u) + f(x) - x$. The $-x$ is the synapse's own decay.

No recurrence ($f = 0$): $\tau\dot{x} = g(u) - x$, so x is just $g(u)$ low-pass filtered by the synapse. It follows the input with time constant τ and forgets it within a few τ once the input goes away.

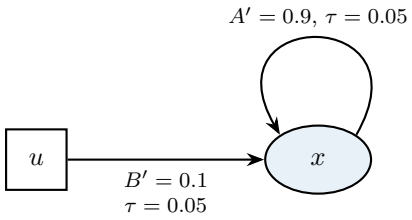
Marking

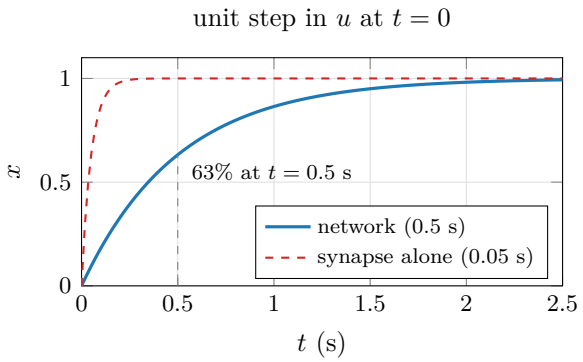
0.25 each for x, g, f, τ (1). 0.5: $-x$ comes from the synapse filter. 0.5: without recurrence x is the synapse low-pass of $g(u)$ and decays to zero after the input stops.

(b) [2] Here $A = -1/0.5 = -2$ and $B = 1/0.5 = 2$. Match $\tau\dot{x} = (A'-1)x + B'u$ to $\tau\dot{x} = \tau Ax + \tau Bu$:

$$A' = \tau A + 1 = 0.05(-2) + 1 = \mathbf{0.9}, \qquad B' = \tau B = 0.05(2) = \mathbf{0.1}.$$

Network: an input node u connects to the population x with `transform=0.1, synapse=0.05`, and x connects to itself with `transform=0.9, synapse=0.05`. The step response has time constant **0.5 s**, which is $\tau/(1 - A') = 0.05/0.1$: the network is 10 times slower than the synapse. It settles at $B'/(1 - A') = 1$ for a unit step.





Marking

0.5 each: $A' = 0.9$, $B' = 0.1$. 0.5: network drawn with both transforms on the right connections. 0.5: step response time constant 0.5 s (not 0.05 s).

Where marks are usually lost

Using $A' = \tau A$ (forgetting the $+1$), or putting $\tau_d = 0.5$ s on the synapse. The synapse stays at 50 ms and the feedback makes the network slower.

Question 3 (4)

(a) [2]

$$\tau\dot{x} = u + (x - x^2) - x = \boxed{u - x^2} = \varphi(x).$$

For $u = 0.25$: $\varphi = 0$ when $x^2 = 0.25$, i.e. $x^* = \pm 0.5$. The slope is $\varphi'(x) = -2x$:

- $x^* = +0.5$: $\varphi' = -1 < 0$, so it is **stable**. φ crosses zero going down: just left of it x increases, and just right of it x decreases.
- $x^* = -0.5$: $\varphi' = +1 > 0$, so it is **unstable**. φ crosses zero going up, and nudges grow.

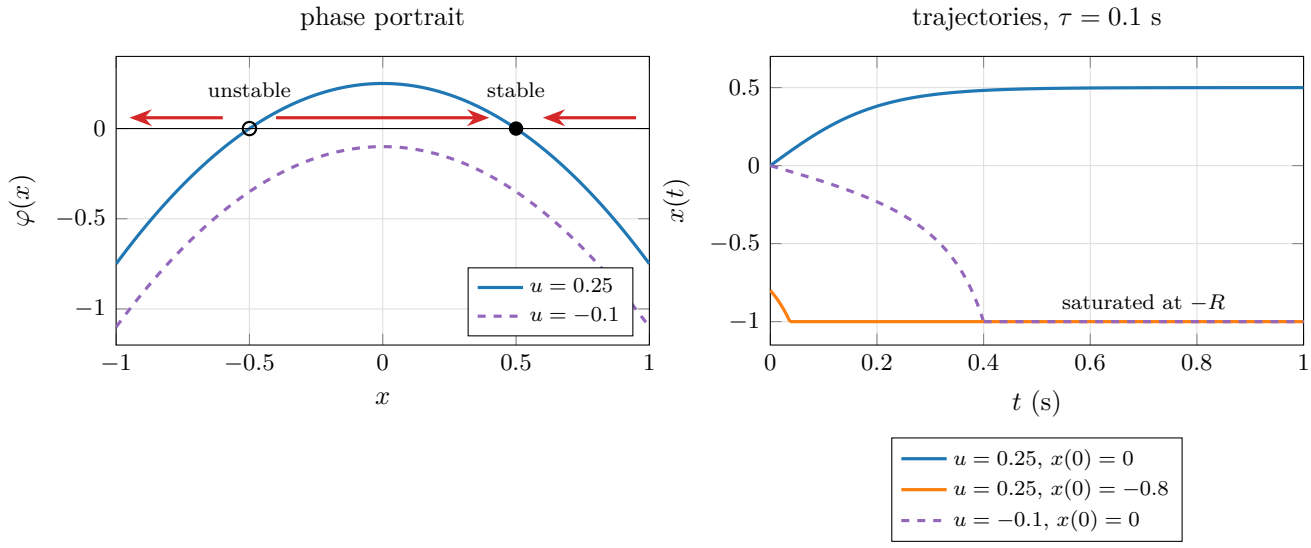
Marking

1: $\tau\dot{x} = u - x^2$ (the $+x$ in f cancels the synapse's $-x$). 0.5: $x^* = \pm 0.5$. 0.5: $+0.5$ stable and -0.5 unstable, with a reason.

(b) [2]

- $x(0) = 0$: $\varphi(0) = 0.25 > 0$, so x rises and settles at $+\mathbf{0.5}$. Exactly, $x(t) = 0.5\tanh(0.5t/\tau)$.
- $x(0) = -0.8$: this is left of the unstable point. $\varphi(-0.8) = 0.25 - 0.64 < 0$, so x **decreases and runs away**. It stops only when the population saturates near -1 (the radius).
- $u = -0.1$: $\varphi = -0.1 - x^2 < 0$ everywhere, so there are **no equilibria**. The parabola has dropped below the axis and the two equilibria have disappeared. From any start x decreases until it saturates near -1.

The plots use $\tau = 0.1$ s. After the state reaches the radius, the trajectories are drawn as saturating at -1.



Marking

0.5: parabola opening down, crossing at ± 0.5, with arrows (right between the roots, left outside them). 0.5: $x(0) = 0$ goes to $+0.5$. 0.5: $x(0) = -0.8$ runs away to the negative saturation. 0.5: $u = -0.1$ has no equilibria and x always decreases.

Where marks are usually lost

Saying $x(0) = -0.8$ goes to -0.5. An unstable point repels, so a start beyond it moves *away*.

Question 4 (4)

(a) [1] $A' = \tau A + I$ with $\tau\omega = 0.1 \cdot 2\pi = 0.2\pi \approx 0.63$:

$$A' = \begin{bmatrix} 1 & -0.2\pi \\ 0.2\pi & 1 \end{bmatrix} \approx \begin{bmatrix} 1 & -0.63 \\ 0.63 & 1 \end{bmatrix}.$$

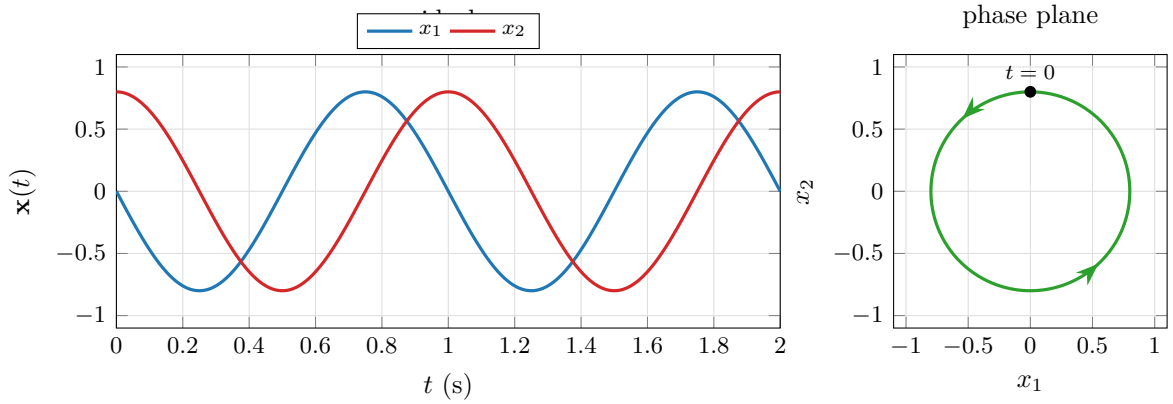
Marking

0.5: identity on the diagonal. 0.5: off-diagonals $\mp\tau\omega = \mp 0.2\pi$ with the signs in the same places as A.

(b) [2] Solve $\dot{x}_1 = -\omega x_2$ and $\dot{x}_2 = \omega x_1$ with $\mathbf{x}(0) = (0, 0.8)$:

$$x_1(t) = -0.8\sin(2\pi t), \qquad x_2(t) = 0.8\cos(2\pi t).$$

These are two sinusoids of amplitude 0.8 and period 1 s (two full cycles in 2 s), 90° apart. At $t = 0$, x_2 is at its peak and x_1 is heading *negative*. In the plane, the state moves around a circle of radius 0.8 **counter-clockwise**: from $(0, 0.8)$ to $(-0.8, 0)$ to $(0, -0.8)$ and back.

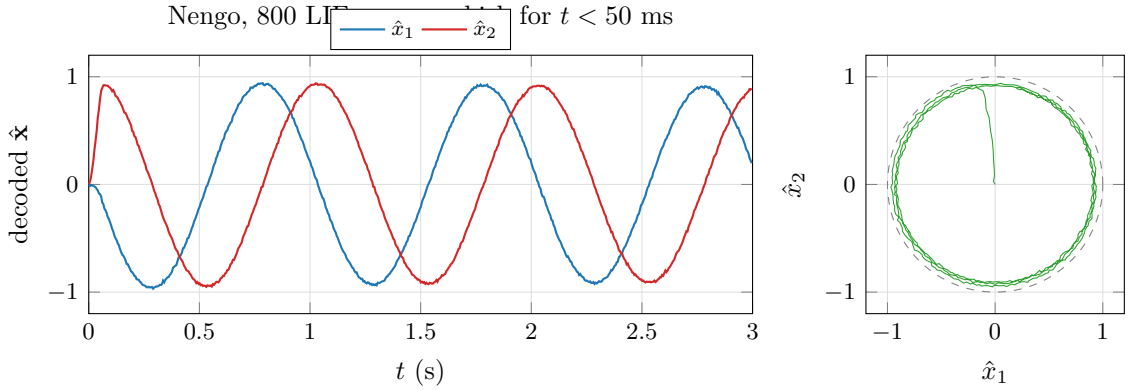


Marking

0.5: both signals have period 1 s and amplitude 0.8. 0.5: 90° apart with the right starting values ($x_1 = 0$, $x_2 = 0.8$). 0.5: circle of radius 0.8. 0.5: counter-clockwise (from $(0, 0.8)$, x_1 goes negative first).

(c) [1] **Kick:** $\mathbf{x} = 0$ is an equilibrium ($A'\mathbf{0} = \mathbf{0}$). Without an input to move it off zero the network sits there forever. The kick sets the initial amplitude.

Amplitude in Nengo: ideally the circle keeps whatever amplitude the kick gave. In practice decoding errors make it slowly grow or shrink, and growth is capped by the **radius**: the neurons saturate, so the decoded value cannot go far past $|\mathbf{x}| \approx 1$. In the simulation below, a strong kick into x_2 makes the oscillator settle on a circle of radius about 0.9.



Marking

0.5: zero is an equilibrium, so it needs a kick. 0.5: amplitude is set by the kick and limited by the radius or saturation (and decoding error).

Question 5 (4)

(a) [2] $x(t) = \int_0^t u\, dt'$:

$$x(t) = \begin{cases} \int_0^t 4t'\, dt' = 2t^2, & 0 \le t < 0.5, \\ 2(0.5)^2 = 0.5, & t \ge 0.5. \end{cases}$$

This is a parabola from 0 to 0.5, followed by a flat hold at 0.5. The slope at $t = 0.5^-$ is $u = 2$, and it drops to 0 instantly when the input stops, so the curve has a *corner* at $t = 0.5$.

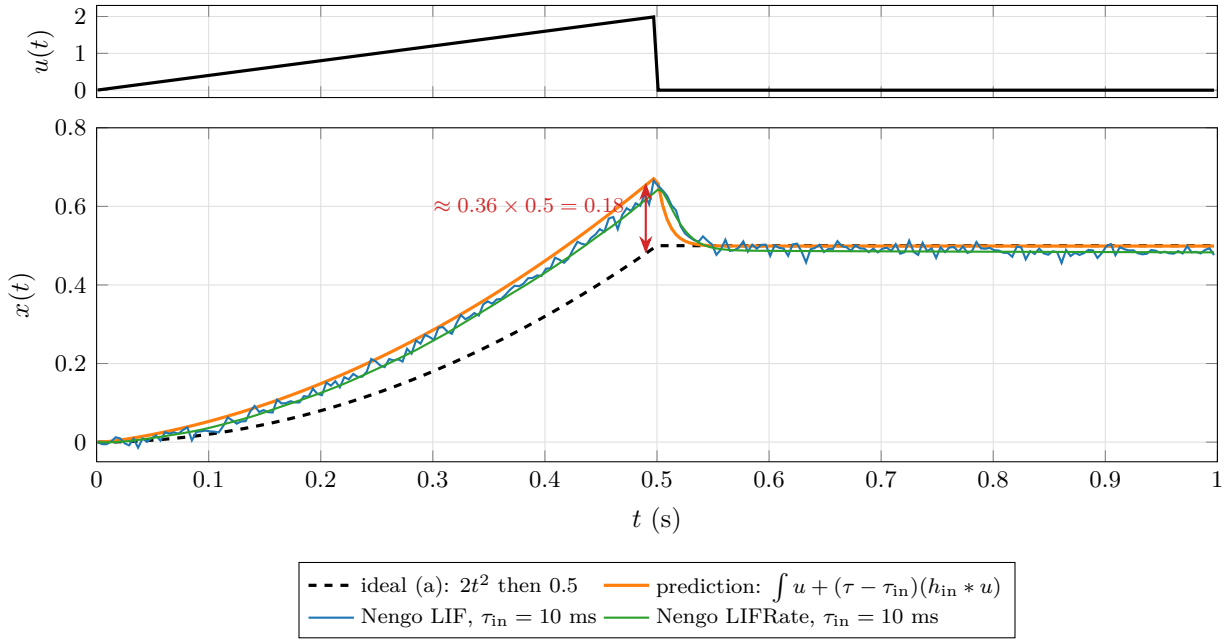
Marking

1: $2t^2$ on $[0, 0.5)$. 0.5: holds at 0.5 after. 0.5: sketch is a parabola then flat (not a straight ramp).

(b) [2] With different synapses on the input and the loop, $X = U/s + (\tau - \tau_{\text{in}})\,U/(1 + s\tau_{\text{in}})$. The decoded value is the true integral **plus an extra term** $(\tau - \tau_{\text{in}})$ **times the (filtered) input**:

$$\hat{x}(t) \approx 2t^2 + (0.1 - 0.01) \cdot 4t = 2t^2 + 0.36t \quad (t < 0.5).$$

- The trace **runs ahead** of the ideal, and the gap grows with the input. Just before $t = 0.5$ s it is about $0.36 \times 0.5 \approx \mathbf{0.18}$, so $\hat{x} \approx 0.68$ instead of 0.5 (the simulation gives about 0.63–0.65, because the probe filter and decoding error shave a little off).
- When the ramp stops, the extra term vanishes within a few τ_{in}. The trace **drops back to** 0.5, so the final value is still correct.



Marking

1: trace above the ideal during the ramp, with a gap that grows (proportional to u). 0.5: gap ≈ 0.18 near $t = 0.5$ (accept 0.15–0.2). 0.5: drops back to 0.5 after the ramp stops (final value unchanged).

Where marks are usually lost

Saying the faster input synapse only adds a small *delay*. The mismatch adds an offset equal to $(\tau - \tau_{\text{in}})u$, which is large here because u reaches 2.

Bonus Question 1 (2)

Why it drifts: an integrator holds its value only if the recurrent connection returns *exactly* x, so that $\tau\dot{x} = f(x) - x = 0$ for every x. With a finite number of neurons the decoded $\hat{f}(x)$ is only approximately x. Wherever $\hat{f}(x) > x$ the value creeps up, and wherever $\hat{f}(x) < x$ it creeps down. The "neural phase portrait" $\hat{f}(x) - x$ is a small wiggly curve rather than zero. The state slides to the nearest point where

that curve crosses zero going down, and stays there. These are the stable fixed points the decoding error creates by accident.

Spiking vs rate: in rate mode the decoding error is a fixed function of x, so the drift is deterministic and smooth (same network, same answer). With spikes, each neuron's output is a train of discrete pulses, so the fed-back signal also carries **spike noise**. The integrator integrates that noise too, which adds a random walk on top of the deterministic drift. Rate mode removes the noise only; the static distortion and its drift are still there.

Marking

1: drift comes from decoding (distortion) error, $\hat{f}(x) \neq x$, which creates fixed points the state slides toward. 1: spike noise is integrated (random walk); rate mode removes the noise but not the distortion.